

Introduction to AWS Step Functions

Ananda Dwi Rahmawati



Hello!

- Sr. Cloud Engineer @ Btech
- AWS CB Q4 2021 - Now
- Tech background: System, Networking, IaaS & PaaS Cloud, DevOps, a bit of Programming
Open Source Enthusiast and Communities Member

<https://linktr.ee/misskecupbung>



AWS Lambda

- Less flexibility for long-running and complex operations
- Other use cases:
 - Retry connection
 - Wait before moving on the next action
 - Run something in parallel



AWS Step Function

State Machine Services





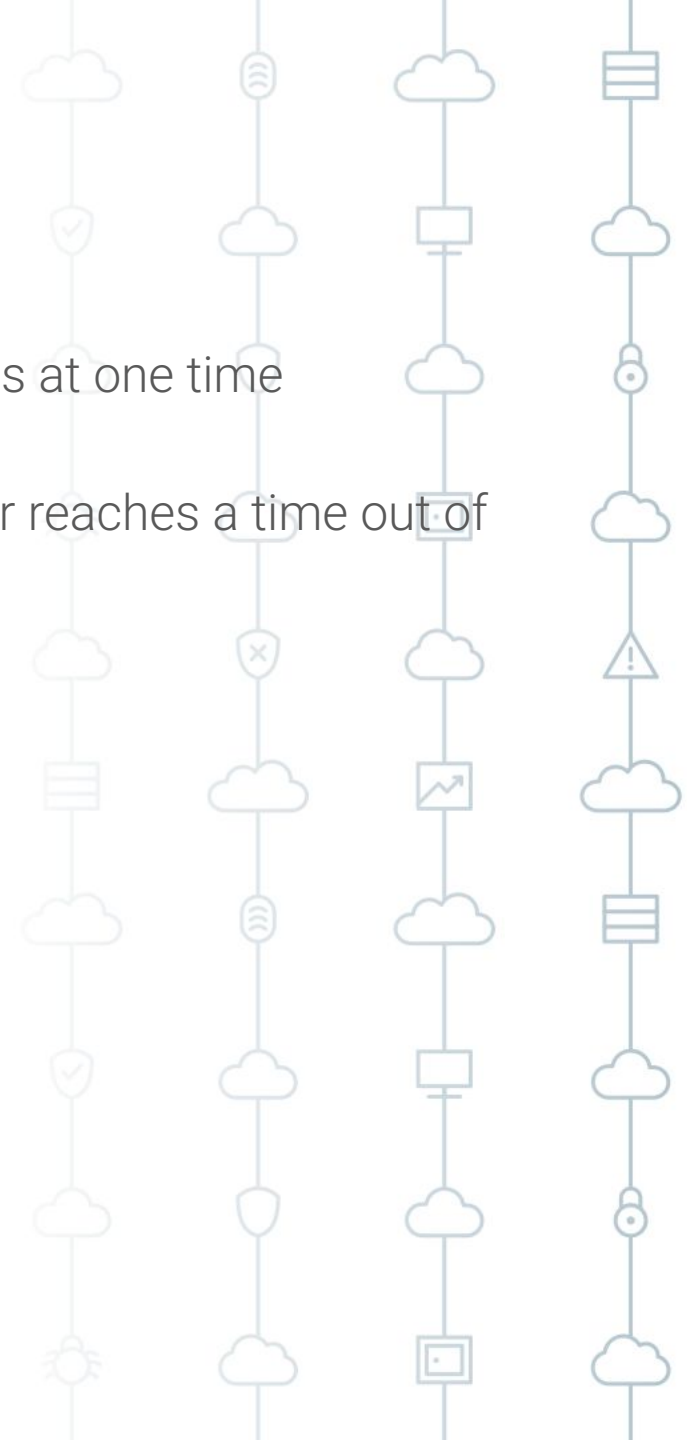
AWS Step Functions

Wait for **inputs**, make **decisions**, and **process** information based on the input variables.

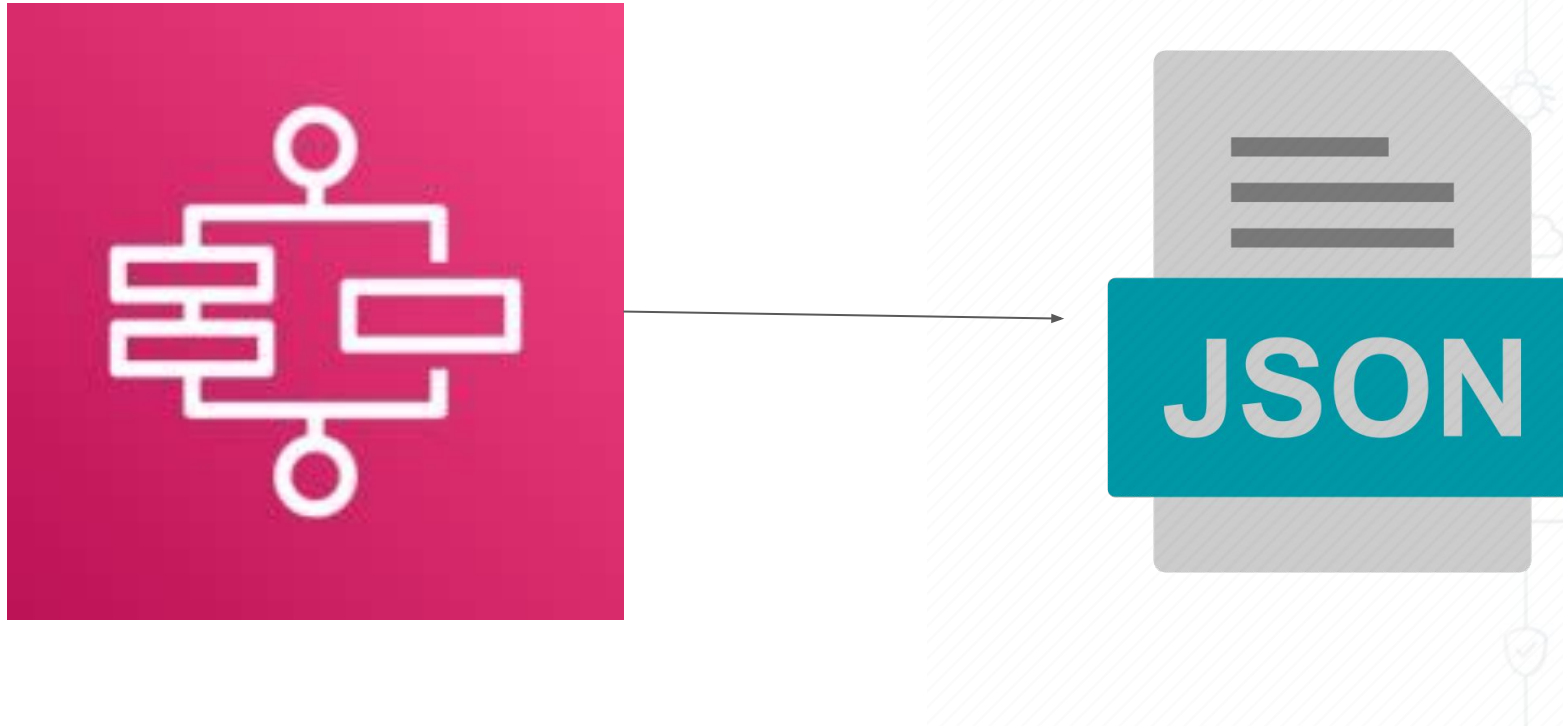


AWS Step Functions

- **In parallel**, for when you have multiple items or tasks you want to process at one time
- **In sequence**, for when order is important.
- **In retry**, maybe you want your code to keep executing until it succeeds, or reaches a time out of some sort.
- **If then**, allows branching and logical trees for decision making.



How Does It Work?



operates by reading in your workflow from an amazon state language file - a **JSON based structured** language used to define your state machine and its various components.

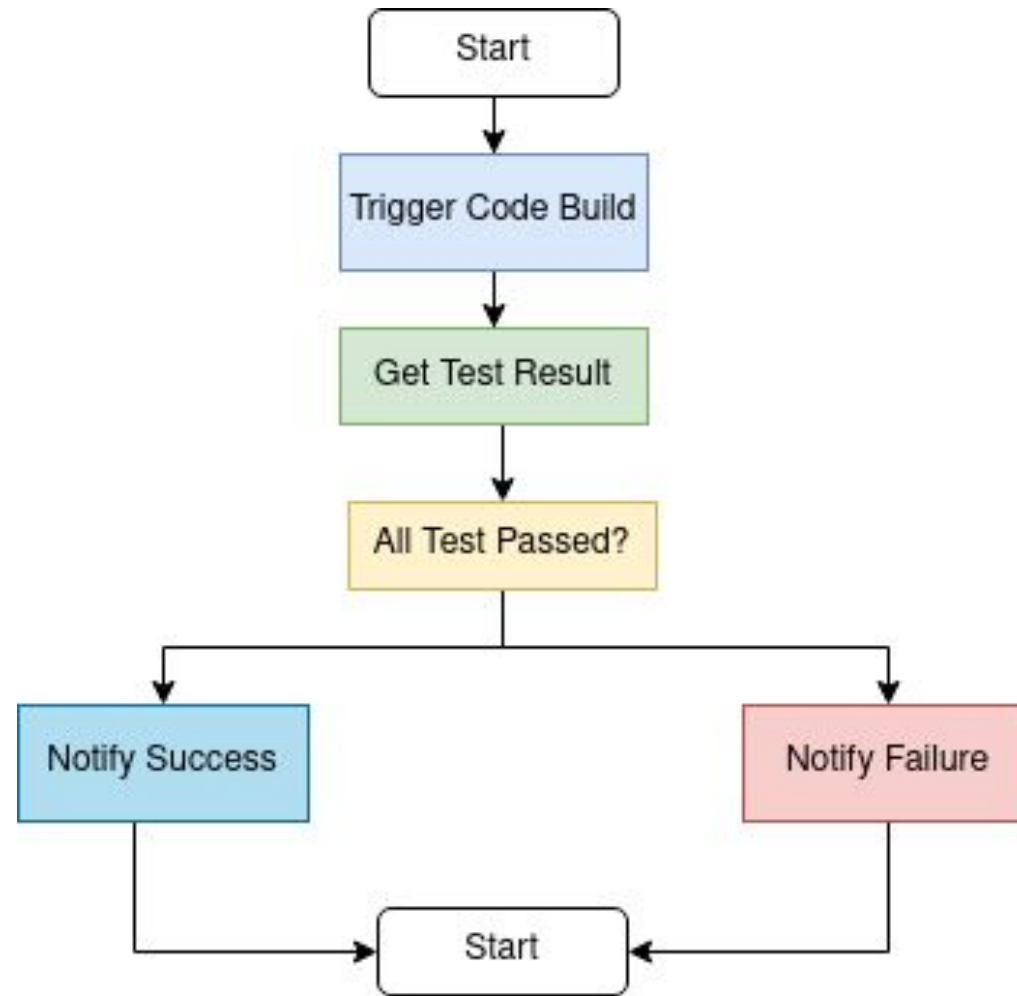
Amazon State Language

- Proprietary language that consists of a collection of states
- Example: <https://states-language.net/>

```
{
  "Comment": "A simple minimal example of the States language",
  "StartAt": "Hello World",
  "States": {
    "Hello World": {
      "Type": "Task",
      "Resource": "arn:aws:lambda:us-east-1:123456789012:function:HelloWorld",
      "End": true
    }
  }
}
```

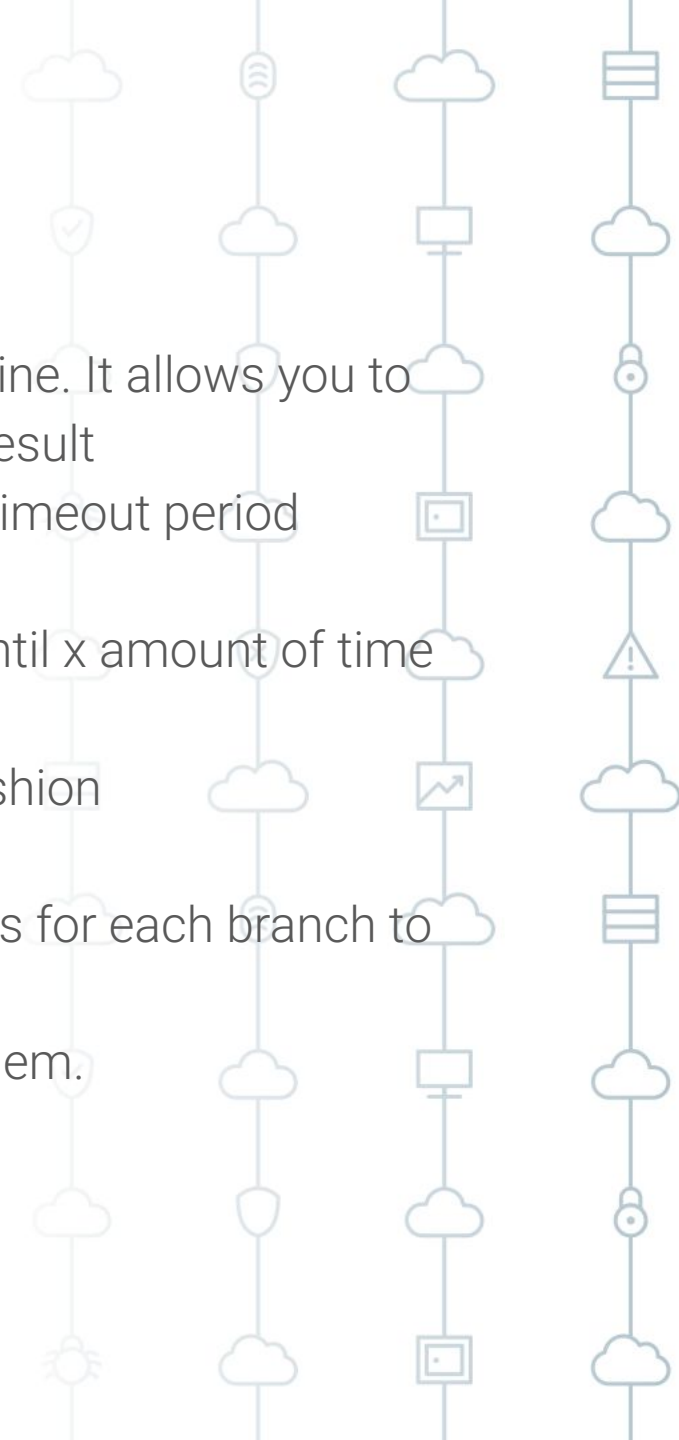
Start > **Hello World** > End

Amazon State Language



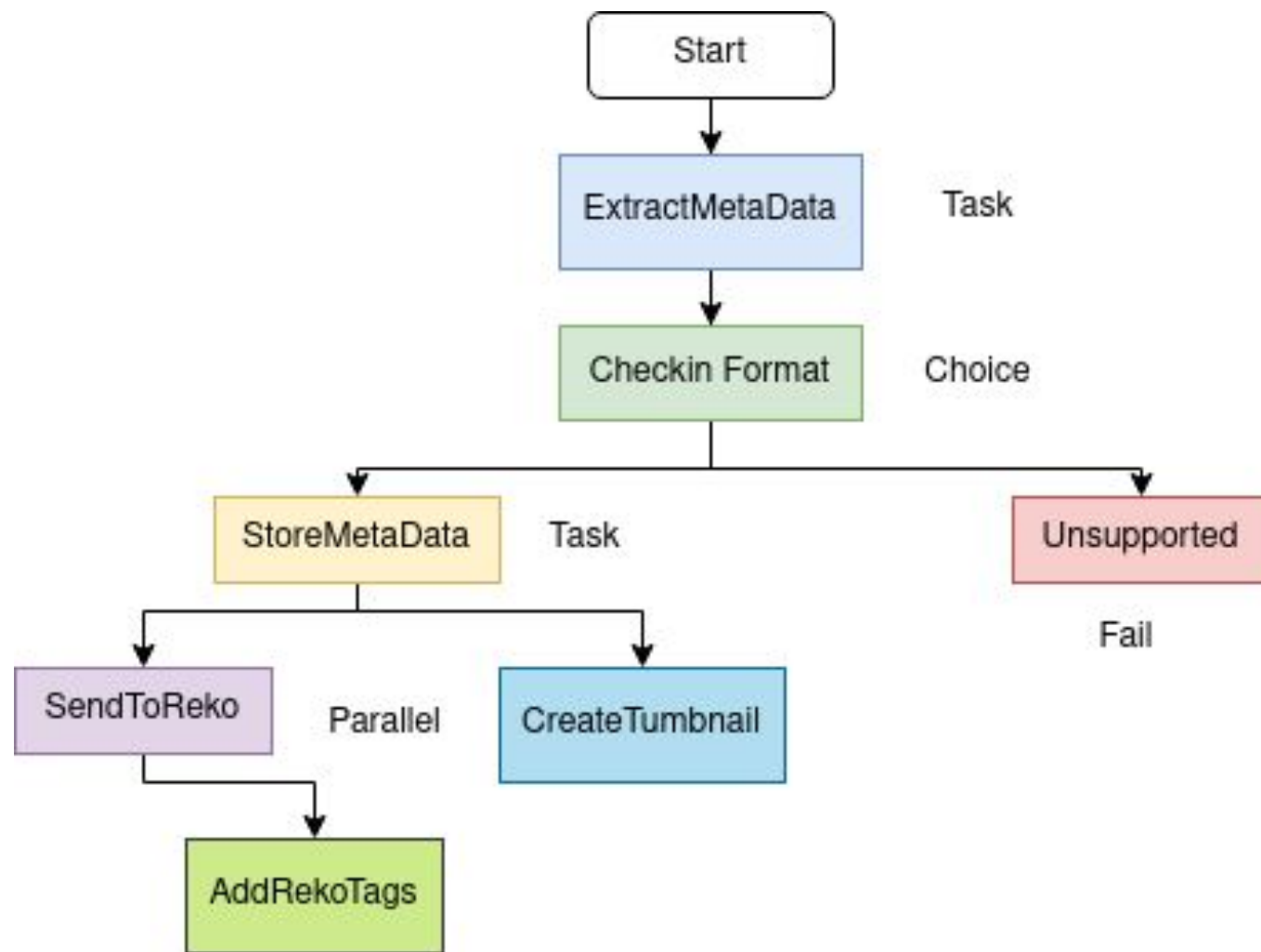
State Types

- **Pass:** a debugging state or one to be used when first creating your machine. It allows you to pass its input value straight through to its output, as well as add a fixed result
- **Task:** You define a resource you wish Step Functions to run as well as a timeout period
- **Choice:** Given an input, the state machine chooses the correct output
- **Wait:** the state machine will pause and can wait until a specific time or until x amount of time has passed
- **Succeed:** Simply the termination of the state machine in a successful fashion
- **Fail:** Termination state for the state machine, in a failed way
- **Parallel:** Executes a group of states as concurrently as possible and waits for each branch to terminate before moving on.
- **Map:** allows you to iterate through a list of items and perform tasks on them.



Using combinations of the states to create your specific state machines - **allows you** to build some very dynamic and impressive serverless solutions that can scale **extremely well!**





State Machine Example

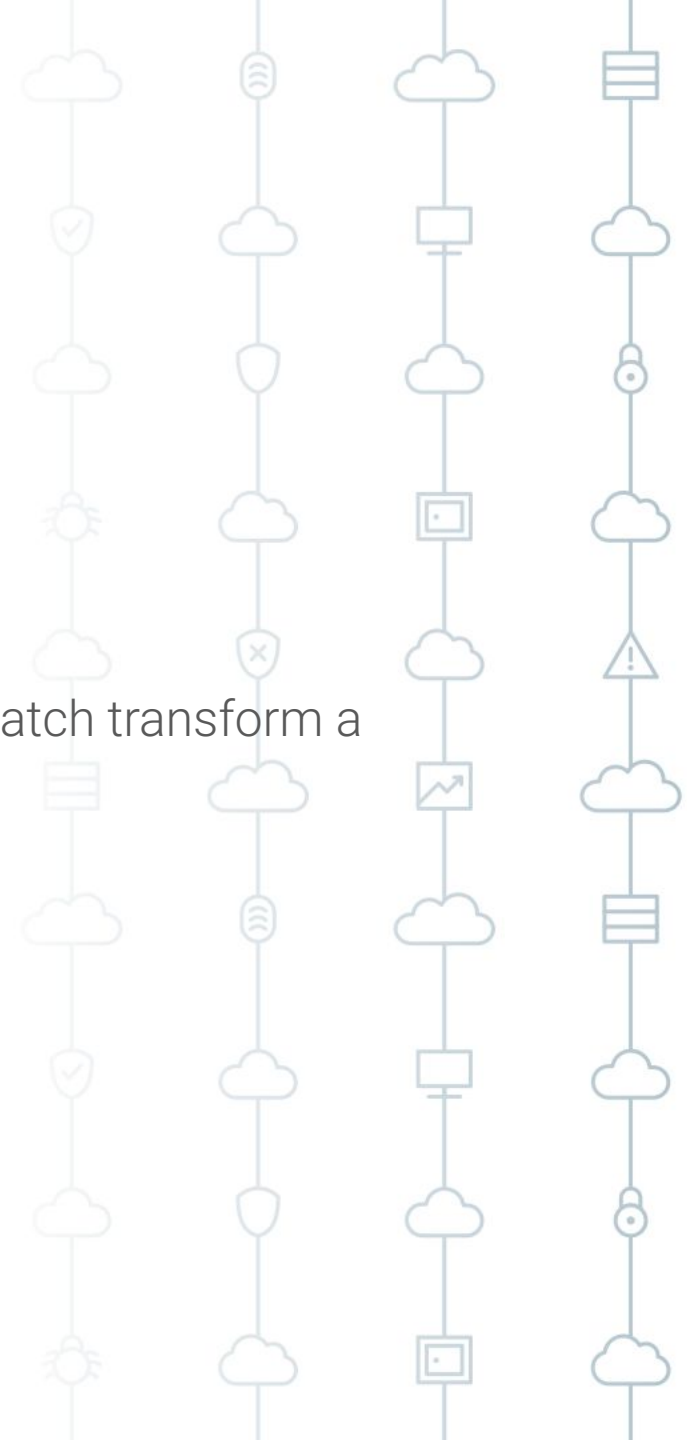


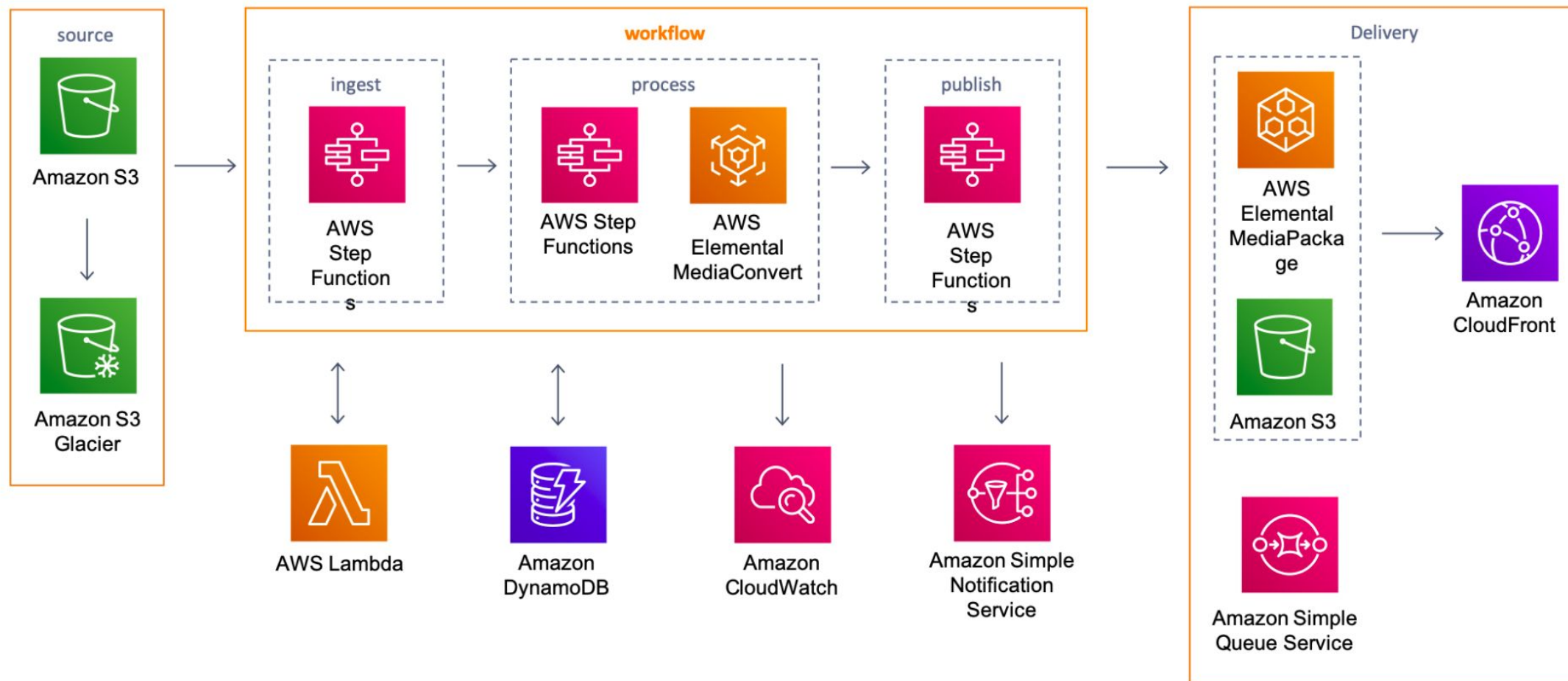
Supported Service Integrations

Service	Response Request	Run a Job (.Sync)	Wait for Callback (.waitForTaskToken)
Lambda	✓		✓
AWS Batch	✓	✓	
DynamoDB	✓		
Amazon ECS/AWS Fargate	✓	✓	✓
Amazon SNS	✓		✓
Amazon SQS	✓		✓
AWS Glue	✓	✓	
Amazon SageMaker	✓	✓	
Amazon EMR	✓	✓	
CodeBuild	✓	✓	
AWS Step Function	✓	✓	✓

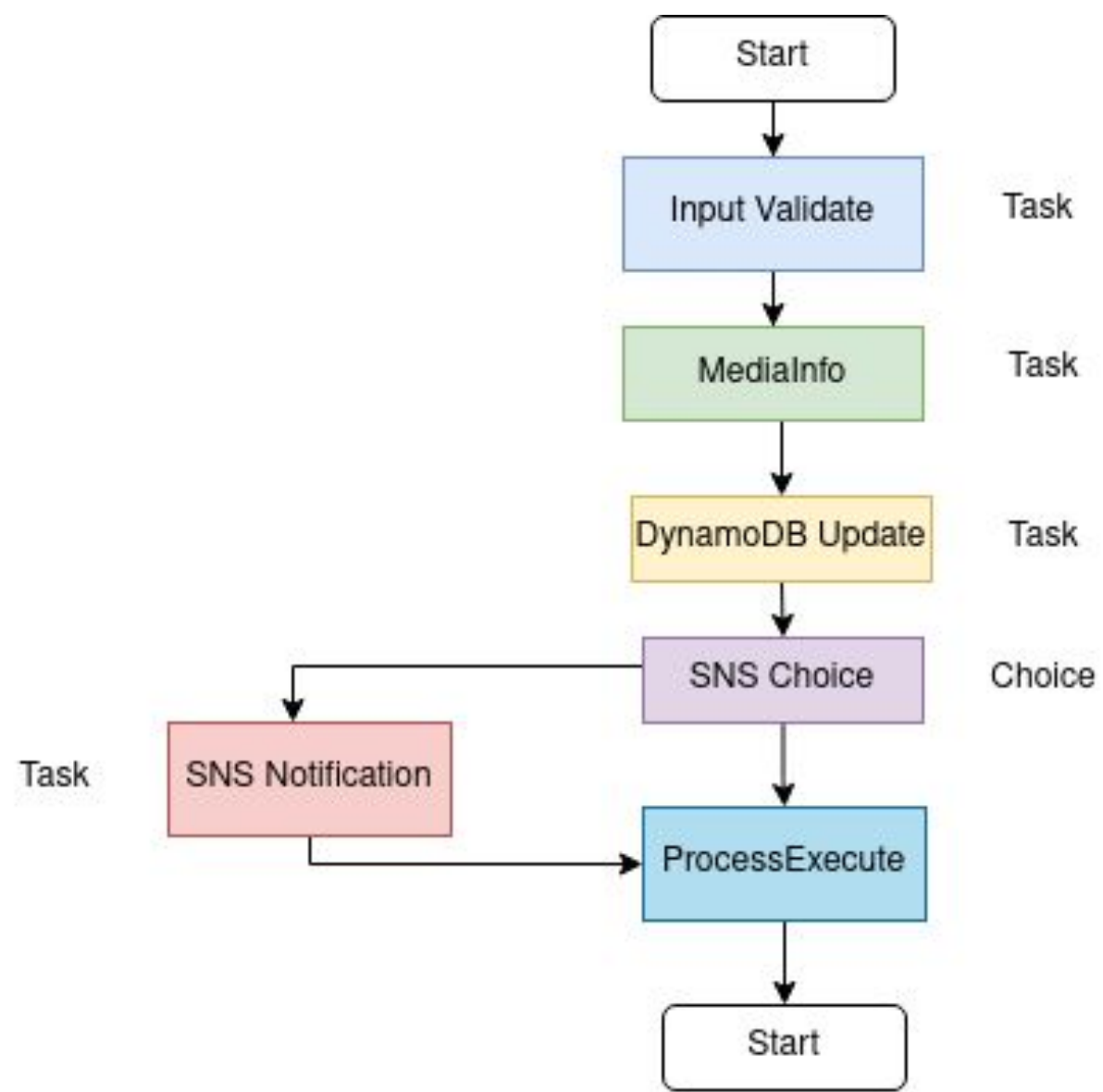
AWS Step Function - Use Cases

- Run an Amazon Elastic Container Service or AWS Fargate task
- Submit an AWS Batch job and wait for it to complete
- Publish a message to an Amazon SNS topic
- Send a message to an Amazon SQS queue
- Start an AWS Glue job run
- Create an Amazon SageMaker job to train a machine learning model or batch transform a dataset





<https://github.com/aws-solutions/video-on-demand-on-aws>



Thank You

